

1. Постановка задачи

Работа выполнена в рамках проекта Desbordante — библиотеки для профилирования данных и поиска зависимостей, в частности точных функциональных зависимостей (ФЗ) вида $X \rightarrow A$.

Требуется расширить библиотеку новым алгоритмом поиска точных ФЗ **FDHits**, основанным на подходе из научной статьи¹: задача поиска ФЗ сводится к построению множества ограничений по нарушающим парам строк и последующему поиску минимальных hitting set в гиперграфе кандидатов. Такой подход эффективен на ряде наборов данных и дополняет уже реализованные в Desbordante методы (Tane, HyFD и др.).

Цель работы — реализовать алгоритм FDHits в кодовой базе Desbordante, интегрировать его в существующую инфраструктуру PLI-индексов и предоставить пользователю возможность запуска алгоритма.

2. Описание предлагаемого решения

В подсистему алгоритмов поиска функциональных зависимостей добавлен новый модуль `src/core/algorithms/fd/fdhits`. Реализация интегрирована в сборку (CMake), общий реестр FD-алгоритмов и Python bindings.

Алгоритм использует PLI-предобработку (через существующий препроцессор HyFD) и далее работает следующим образом: по данным формируется гиперграф ограничений из нарушающих пар строк, затем выполняется древовидный поиск минимальных hitting set (**SearchDE** для фиксированного RHS и **SearchMulti** для объединённой постановки). Корректность кандидатов проверяется PLI-валидатором **PLIValidator**; при обнаружении контрпримера гиперграф дообогачается новыми рёбрами, а пересечения PLI переиспользуются через кэш **PLICache**. Найденные зависимости преобразуются к формату Desbordante и регистрируются в общем списке результатов.

3. Апробация

Апробация реализованного алгоритма FDHits выполнена в форме сравнительных бенчмарков с алгоритмами, уже доступными в Desbordante: Tane и HyFD (в однопоточном и многопоточном режимах).

¹Tobias Bleifuß, Thorsten Papenbrock, Thomas Bläsius, Martin Schirneck, and Felix Naumann. 2024. Discovering Functional Dependencies through Hitting Set Enumeration. <https://doi.org/10.1145/3639298>

3.1. Условия эксперимента

Эксперимент проводился при следующих параметрах: MacBook Pro (Mac14,10), Apple M2 Pro (12 cores), 16 GB RAM, macOS 26.3; CMake 4.2.3; Apple clang 17.0.0 (arm64-apple-darwin25.3.0); Boost 1.85; build type Release. Для каждого датасета выполнялось 5 прогонов; фиксировалось wall-clock время выполнения, в таблице приведены min/max и среднее (Avg).

3.2. Результаты бенчмарков

Таблица 1: Сравнение времени работы алгоритмов поиска ФЗ

Dataset	Algorithm	Avg Time, s	Min, s	Max, s	FDs
fd-reduced-30	Tane	112.0860	105.1603	116.0376	89571
fd-reduced-30	HyFD-1t	123.0098	119.2914	125.8773	89571
fd-reduced-30	HyFD-8t	18.7195	18.1070	19.6888	89571
fd-reduced-30	FDHits	6.1519	6.0310	6.3031	89571
adult	Tane	39.7578	37.4562	41.4142	78
adult	HyFD-1t	0.1376	0.1351	0.1400	78
adult	HyFD-8t	0.1283	0.1263	0.1320	78
adult	FDHits	0.0314	0.0265	0.0345	78
EpicMeds	Tane	43.4844	42.7337	44.2654	16
EpicMeds	HyFD-1t	6.7129	6.6456	6.8453	16
EpicMeds	HyFD-8t	3.8282	3.7991	3.8829	16
EpicMeds	FDHits	0.3999	0.3853	0.4213	16

Во всех экспериментах число найденных функциональных зависимостей совпадает между алгоритмами для каждого датасета, а FDHits демонстрирует наименьшее время выполнения.

4. Заключение

В ходе работы реализован и интегрирован в Desbordante алгоритм FDHits для поиска точных функциональных зависимостей. Основные результаты:

- добавлен модуль `FDHits` и подключён к сборке (CMake) и реестру FD-алгоритмов;
- реализованы режимы `union` и `per_target` (опция `fdhits_mode`);
- обеспечен запуск алгоритма из Python через bindings;
- выполнены бенчмарки, подтверждающие высокую производительность на выбранных датасетах.

Реализация алгоритма находится на ревью. Pull request #701 доступен на GitHub.